

TDS200 · Kryssplattform

Session 8

Authentication: Email/Password & Context

Authentication · Context · Auth State · Router Guard

Cross-Platform Mobile Development

What you'll learn today

- How Firebase Authentication manages user identity
- Email/password sign-up and login
- Share auth state app-wide with React Context
- A custom `useAuth ()` hook (no prop drilling)
- How to enforce router guard



Guarded app

Unauthenticated users are redirected to the login screen.



Project step

Add email/password to your app.

The 12 sessions

1

Introduction & Foundations

2

Components & Navigation (Expo Router)

3

Components, State & Local Storage

4

Forms, Modals & Creating Data

5

Firebase Firestore CRUD

6

Firestore CRUD & Camera

7

Firebase Storage, Camera & Image Picker

8

Firebase Authentication & Context

9

Google sign-in, Firebase Rules & Guards

10

NativeWind & Geolocation

11

Google Maps & Queries

12

Exam & Wrap-Up

What is new in the codebase?

- We have **removed** `app/(tabs)/authenticationPage.tsx`.
- We have **added** `app/auth.tsx` and modified the root layout `app/_layout.tsx`, so that authentication can control the whole app while login screen stays outside of the tab navigation.
- We have **added** `context/AuthContext.tsx` to manage authentication globally.
- We have **removed** `Utils/userData.ts` since we now use Firebase authentication.
- We have **added** `authApi.ts` under the `api` folder to manage Firebase authentication.
- We modify the files app-wise to get authentication state directly from Firebase, i.e., **`postDetails/[id].tsx`**, **`index.tsx`**, **`profilePage.tsx`**.
- We have modified **`profilePage.tsx`** to handle user sign-out from Firebase.

How a 2-hour session runs

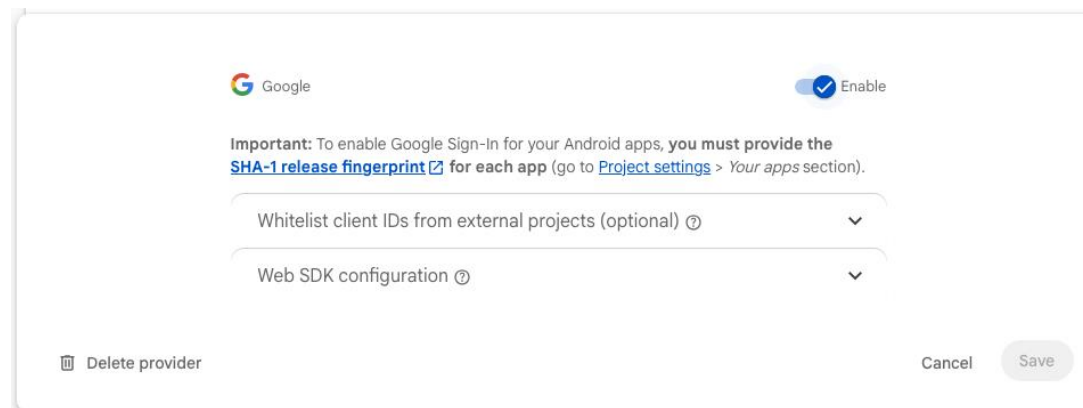
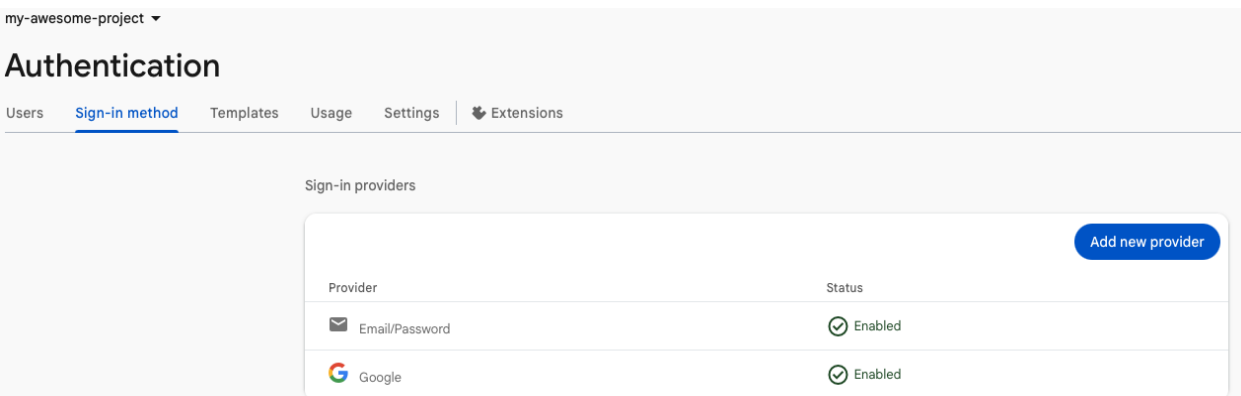
0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Auth concepts & email/password Sign-up, login, sign-out
0:45-1:00	Break —
1:00-1:40	Context & the auth provider Provider, consent, redirect URIs
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

What Firebase Auth manages for you

- **Securely stores credentials and issues a session token.**
 - Many providers: email/password, Google, Apple, GitHub, anonymous...
 - Each signed-in user gets a unique uid — your key for ownership.
 - `onAuthStateChanged` tells the app when sign-in state changes.
- **OAuth 2.0 (Google): the user authenticates with Google, not you.**
 - You never see their password; you receive a verified identity.
 - We will introduce Google sign-in in later session as extended feature.
- **In Sign-in method, you can enable/disable different method.**

SECURITY

Never store raw passwords yourself.
Let the provider do it.



SETUP + AUTHENTICATION

Email / password auth

firebaseConfig.ts

```
export const auth = createAuth();
```

- **Step 1:** Firebase will provide getAuth API for you to use authentication services, the same as Firestore and Storage. Update firebaseConfig.ts.
 - Firebase/firestore → getFirestore
 - Firebase/storage → getStorage
 - Firebase/auth → getAuth (we use **createAuth** since we want to handle Web and simulators)
- **Step 2:** Enable Email/Password in Firebase console → Authentication → enable Sign-in method.
- **Step 3:** at client app code,
 - Import auth from firebaseConfig.
 - import and use createUserWithEmailAndPassword to register.
 - Import and use signInWithEmailAndPassword to log in.
 - Import and use signOut to log out.

authApi.ts

```
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword, signOut,
} from 'firebase/auth';
import { auth } from './firebaseConfig';

export const register = (e: string, p: string) =>
  createUserWithEmailAndPassword(auth, e, p);

export const login = (e: string, p: string) =>
  signInWithEmailAndPassword(auth, e, p);

export const logout = () => signOut(auth);
```

SETUP + AUTHENTICATION

firebaseConfig

- **Why persistence?** Without persistence, users would be logged out every time the app restarts.
- **Firebase Authentication stores user sessions differently on web and native platforms.**
 - Web browsers already have built-in storage support, so `getAuth(app)` automatically uses browser persistence (e.g., Local Storage).
 - For native React Native apps (including iOS simulator, Android emulator, and physical devices), there is no browser storage, so Firebase needs to be explicitly configured to use `AsyncStorage` by `getReactNativePersistence()`.
- We then export `auth=createAuth()` for app-wide usage.

firebaseConfig.ts

```
import { getAuth } from 'firebase/auth';

function createAuth() {
  if (Platform.OS === "web") {
    return getAuth(app);
  }

  const { getReactNativePersistence } = require(
    "@firebase/auth/dist/rn/index.js");
  return initializeAuth(app, {
    persistence: getReactNativePersistence(AsyncStorage)
  });
}

export const auth = createAuth();
```

SETUP + AUTHENTICATION

Fix Typescript error

- Two ways to require @firebase/auth/dist/rn/index
 - tsConfig configuration
 - Direct require from firebaseConfig.
- Why do we need this? To fix TypeScript error.

firebaseConfig.ts

```
import { getAuth } from 'firebase/auth';
function createAuth() {
  if (Platform.OS === "web") {
    return getAuth(app);
  }

  Const { getReactNativePersistence } = require(
    "@firebase/auth/dist/rn/index.js");
  return initializeAuth(app, {
    persistence: getReactNativePersistence(AsyncStorage)
  });
}

export const auth = createAuth();
```

firebaseConfig.ts

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getStorage } from "firebase/storage";
import { getAuth, initializeAuth, getReactNativePersistence } from "firebase/auth";
import { Platform } from "react-native";
import AsyncStorage from "@react-native-async-storage/async-storage";

> const firebaseConfig = { ...
};
const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
// storage is used by imageApi to upload and download image files.
export const storage = getStorage(app);

// Web keeps its default browser persistence; native stores the login in AsyncStorage.
function createAuth() {
  if (Platform.OS === "web") {
    return getAuth(app);
  }
  return initializeAuth(app, {
    persistence: getReactNativePersistence(AsyncStorage),
  });
}
export const auth = createAuth();
```

tsConfig.ts

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "jsx": "react",
    "strict": true,
    "paths": {
      "@/*": ["./*"],
      "@firebase/auth": [".node_modules/@firebase/auth/dist/index.rn.d.ts"]
    },
    "include": ["**/*.ts", "**/*.tsx", ".expo/types/**/*.ts", "expo-env.d.ts"]
  }
}
```

authApi

All Firebase Auth calls are placed in authApi, so that screen pages do not have handle Firebase related logic.

```
import {
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  updateProfile,
  signOut,
} from "firebase/auth";
import { auth } from "@firebaseConfig";

export async function signInWithEmail(email: string, password: string) {
  return signInWithEmailAndPassword(auth, email, password);
}

export async function signUpWithEmail(email: string, password: string, name: string) {
  const { user } = await createUserWithEmailAndPassword(auth, email, password);
  await updateProfile(user, { displayName: name });
  return user;
}

export async function signOutUser() {
  return signOut(auth);
}
```

Auth.tsx

The authentication page handles sign up and sign in by calling functions defined in authApi.ts

```
async function handleSignIn() {  
  if (!email.trim() || !password) {  
    showToast("Please enter your email and password.");  
    return;  
  }  
  setLoading(true);  
  try {  
    await signInWithEmail(email.trim(), password);  
  } catch (e: any) {  
    showToast(getSignInErrorMessage(e.code));  
  } finally {  
    setLoading(false);  
  }  
}  
  
async function handleSignUp() {  
  if (!name.trim()) { showToast("Please enter your name."); return; }  
  if (!email.trim()) { showToast("Please enter your email."); return; }  
  if (password.length < 6) { showToast("Password must be at least 6 characters."); return; }  
  
  setLoading(true);  
  try {  
    await signUpWithEmail(email.trim(), password, name.trim());  
  } catch (e: any) {  
    showToast(getSignUpErrorMessage(e.code));  
  } finally {  
    setLoading(false);  
  }  
}
```

Authentication

- To implement Firebase authentication in our expo app, three pieces work together:
 - **Firebase Auth** determines whether a user is signed in.
 - **React Context** stores the current authentication state and makes it available throughout the app.
 - **Expo Router** uses that authentication state to decide which screens the user can access.

onAuthStateChanged

- **Firestore Auth stores the signed-in session on the device.** When the app reopens, Firestore restores the previous authentication session and determines whether a user is still signed in. The user normally does not need to sign in again every time when the app restarts.
- `onAuthStateChanged()` listens to the **authentication state change**.

```
useEffect(() => {  
  // onAuthStateChanged fires once on startup (after Firestore reads the persisted session  
  // from AsyncStorage on iOS or localStorage on web) and again on every sign-in / sign-out.  
  const unsubscribe = onAuthStateChanged(auth, (firebaseUser) => {  
    setUser(firebaseUser);  
    setLoading(false);  
  });  
  return unsubscribe;  
}, []);
```

onAuthStateChanged

- You only register `onAuthStateChanged()` once, i.e., when the `AuthProvider` is mounted. Firebase then calls the listener:
 - When the app starts and the existing authentication state has been restored.
 - When a user signs in, and
 - When a user signs out.
- The listener returns either
 - A Firebase User object when someone is signed in, or
 - Null if no user is signed in.

```
useEffect(() => {  
  // onAuthStateChanged fires once on startup (after Firebase reads the persisted session  
  // from AsyncStorage on iOS or localStorage on web) and again on every sign-in / sign-out.  
  const unsubscribe = onAuthStateChanged(auth, (firebaseUser) => {  
    setUser(firebaseUser);  
    setLoading(false);  
  });  
  return unsubscribe;  
}, []);
```

Why Context for auth

- Auth state is needed in many parts of the app — passing it through multiple levels of props is painful.
- React Context provides **shared authentication state**. Any component inside the provider can access the current user without receiving it through props.
- An **AuthProvider** wraps the app and exposes values such as
 - user
 - loading
 - other authentication-related state or functions

```
// 1. Define what data the context will provide
type AuthContextType = {
  user: User | null;
  isLoading: boolean;
};
// 2. Create the shared context
const AuthContext = createContext< AuthContextType | null>(null);

// 3. AuthProvider sets up the onAuthStateChanged listener
//and makes user and isLoading available to every component inside this provider.
export function AuthProvider({ children }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => onAuthStateChanged(auth, (user) => {
    setUser(user); setIsLoading(false);
  }), []);

  return <AuthContext.Provider value={{ user, isLoading}}>
    {children}
  </AuthContext.Provider>;
}

// 4. useAuth is the custom hook any component calls to read user + isLoading
export const useAuth = () => useContext(AuthContext)!;
```

Why Context for auth

- A custom hook such as **useAuth ()** makes accessing this context simple:

```
const {user, loading} = useAuth();
```
- This gives us one central place to listen to Firebase authentication state changes.
- Instead of calling `onAuthStateChanged()` everywhere, we register it once inside `AuthProvider`, store the result in `Context`, and let the rest of the app read that shared state.
- This avoids duplicated authentication logic and unnecessary local storage loading.

```
// 1. Define what data the context will provide
type AuthContextType = {
  user: User | null;
  isLoading: boolean;
};
// 2. Create the shared context
const AuthContext = createContext< AuthContextType | null>(null);

// 3. AuthProvider sets up the onAuthStateChanged listener
//and makes user and isLoading available to every component inside this provider.
export function AuthProvider({ children }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => onAuthStateChanged(auth, (user) => {
    setUser(user); setIsLoading(false);
  }), []);

  return <AuthContext.Provider value={{ user, isLoading}}>
    {children}
  </AuthContext.Provider>;
}

// 4. useAuth is the custom hook any component calls to read user + isLoading
export const useAuth = () => useContext(AuthContext)!;
```

The AuthSession provider

providers/AuthProvider.tsx (a simplified version)

```
// 1. Define what data the context will provide
type AuthContextType = {
  user: User | null;
  isLoading: boolean;
};
// 2. Create the shared context
const AuthContext = createContext< AuthContextType | null>(null);

// 3. AuthProvider sets up the onAuthStateChanged listener
//and makes user and isLoading available to every component inside this provider.
export function AuthProvider({ children }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => onAuthStateChanged(auth, (user) => {
    setUser(user); setIsLoading(false);
  }), []);

  return <AuthContext.Provider value={{ user, isLoading }}>
    {children}
  </AuthContext.Provider>;
}

// 4. useAuth is the custom hook any component calls to read user + isLoading
export const useAuth = () => useContext(AuthContext)!;
```

- AuthContext.Provider is mostly an internal implementation detail inside your authentication layer. Your UI component normally should not use it directly.
- The public interface to the rest of the app is usually `<AuthProvider>`

The AuthSession consumer

- In a screen or UI component, use `useAuth()` to access the current authentication state.
- You don't need to import `auth` from `firebaseConfig` or read `auth.currentUser` directly in every screen.
- The authentication state is already managed centrally by `AuthProvider` and shared through React Context.
- Because the context is reactive, when the authentication state changes, every component using `useAuth()` receives the new value and re-renders automatically.
- For example, if the user signs out:
 - Firebase Auth state changes
 - `AuthProvider` updates user
 - Context value changes
 - Component using `useAuth()` and re-render

consume anywhere

```
import { useAuth } from "@context/AuthContext";

export default function HomeScreen() {
  const { user } = useAuth();
  const authorName = user?.displayName ?? user?.email
    ?? "Anonymous";
  // ...
}
```

THEORY • CONTEXT

RootNavigator & RootLayout

- Previously, RootLayout directly defines the stack navigation.
- After introducing authentication context, we wrap the entire app inside AuthProvider so that all child components can access the authentication state using useAuth().

```
// RootNavigator is a separate component so it can call useAuth().
// useAuth() only works inside a component that is a child of AuthProvider.
function RootNavigator() {
  const { user, loading } = useAuth();

  const router = useRouter();
  const segments = useSegments();

  useEffect(() => {
    if (loading) return;

    const onAuthScreen = segments[0] === "auth";

    if (!user && !onAuthScreen) {
      router.replace("/auth");
    } else if (user && onAuthScreen) {
      router.replace("/");
    }
  }, [user, loading, segments]);

  if (loading) {
    return (
      <View style={styles.loading}>
        <ActivityIndicator size="large" color="#007AFF" />
      </View>
    );
  }

  return (
    <Stack>
      <Stack.Screen name="(tabs)" options={{ headerShown: false, headerBackTitle: "" }} />
      <Stack.Screen name="auth" options={{ headerShown: false }} />
      <Stack.Screen name="postDetails/[id]" options={{ headerBackTitle: "Back" }} />
    </Stack>
  );
}

// RootLayout wraps everything in AuthProvider so every screen can call useAuth().
export default function RootLayout() {
  return (
    <AuthProvider>
      <RootNavigator />
    </AuthProvider>
  );
}
```

Segments

- Segments comes from Expo routers useSegments hook. It represents the current route as an array of route segments. With Expo router, route groups like (tabs) can appear in the segments array even though they are not part of the URL.
 - app/postDetails/123 will give ["postDetails","123"].
 - app/(tabs)/index.tsx will give ["(tabs)"].
 - app/auth.tsx will give ["(auth)"].
- This allows the guard check `segments[0] === "auth"` without hardcoding the full path.

```
function RootNavigator() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const segments = useSegments();

  useEffect(() => {
    if (loading) return; // wait until Firebase has read the session

    const onAuthScreen = segments[0] === "auth";

    if (!user && !onAuthScreen) {
      router.replace("/auth"); // not signed in → go to auth screen
    } else if (user && onAuthScreen) {
      router.replace("/"); // signed in but on auth screen → go home
    }
  }, [user, loading, segments]);

  if (loading) {
    return <ActivityIndicator />; // show spinner while Firebase loads
  }

  return (
    <Stack>
      <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
      <Stack.Screen name="auth" options={{ headerShown: false }} />
      <Stack.Screen name="postDetails/[id]" options={{ headerBackTitle: "Back" }} />
    </Stack>
  );
}
```

RootNavigator enforces sign-in

- The redirect log is as below:
 - User=null & all other screens except auth: user must sign in, push to /auth
 - User=null & auth screen: already on auth screen, do nothing
 - User=user & auth screen: user just signed in, push to home /
 - User=user & all other screens except auth: already inside app, do nothing

```
function RootNavigator() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const segments = useSegments();

  useEffect(() => {
    if (loading) return; // wait until Firebase has read the session

    const onAuthScreen = segments[0] === "auth";

    if (!user && !onAuthScreen) {
      router.replace("/auth"); // not signed in → go to auth screen
    } else if (user && onAuthScreen) {
      router.replace("/"); // signed in but on auth screen → go home
    }
  }, [user, loading, segments]);

  if (loading) {
    return <ActivityIndicator />; // show spinner while Firebase loads
  }

  return (
    <Stack>
      <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
      <Stack.Screen name="auth" options={{ headerShown: false }} />
      <Stack.Screen name="postDetails/[id]" options={{ headerBackTitle: "Back" }} />
    </Stack>
  );
}
```

RootNavigator enforces sign-in

- Auth.tsx now sits as a sibling of (tabs), not inside it, for several main reasons.
 - Any screen places inside (tabs) gets a tab bar, as part of Tabs navigator. Sign-in/Sign-out is a full screen flow, and it should not have a persistent Home/Profile/Auth tab bar at the bottom.
 - Conceptually, the authentication page belongs outside the authenticated app. (tab) should only represent the screens after sign-in.
- In this way, we enforce sign-in for the whole app, which we can. As router guard.

```
✓ api
  TS authApi.ts
  TS commentApi.ts
  TS imageApi.ts
  TS postApi.ts
✓ app
  ✓ (tabs)
    🌀 _layout.tsx
    🌀 index.tsx
    🌀 profilePage.tsx
    > postDetails
    🌀 _layout.tsx
    🌀 auth.tsx
```

Profile Page

- When we sign out user from profilePage, the system will direct back to the home page using `router.replace("/")` without requiring re-authentication.
- Note that we navigate to Home first, then sign out user. Why?
- If we sign out user first, the auth state changes immediately while we are still on the current profilePage screen, and `_layout.tsx` redirect sends us to `/auth` before we get a chance to navigate to home.
- The order matters!

Sign out user and redirect back to home (profilePage.tsx)

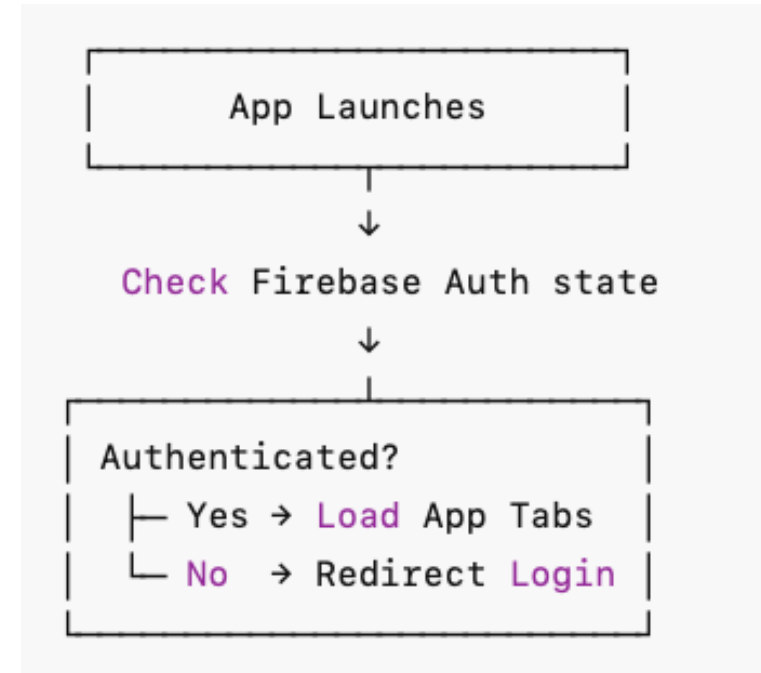
```
export default function ProfilePage() {
  // useAuth() reads from AuthContext – reactive, re-renders if auth state changes.
  // user is guaranteed non-null here because _layout.tsx guards this screen.
  const { user } = useAuth();
  const router = useRouter();
  if (!user) return null;

  const initial = (user.displayName ?? user.email ?? "?").charAt(0).toUpperCase();

  async function handleSignOut() {
    // Navigate to Home first, then sign out. If we sign out first, the auth
    // state change fires while we're still on this profile screen, and
    // _layout.tsx's redirect effect sends us to /auth before this function
    // gets a chance to navigate anywhere itself.
    router.replace("/");
    await signOutUser();
  }
}
```

Router guard

- A router guard can be regarded as a gatekeeper for the app
- A router guard checked whether the user is authenticated before deciding which screen they are allowed to see.
- In our Expo app, user authentication is required for the whole app, which means that an unauthenticated user is redirected to the sign-in screen before accessing any part of the app.



Public Routes vs Protected Routes

- Take a look at a different app setting. In this new version, authentication is not required for the whole app.
- **Public route:**
 - Home page
 - Accessible by everyone (both guest users and signed-in users)
- **Protected routes**
 - Post Details
 - Create new post
 - Accessible only by signed-in users
- If a guest users tries to access a protected route, the app redirects the user to the sign-in page.
- The router guard therefore needs to check:
 - Is the user signed in?
 - Is the current route protected?

Previously, the whole app is protected

Guest user
↓
Sign-in page
↓
Main app

Now, only selected routes are protected

Guest user ———— { Home → allowed
 | Details → sign in required
 | Create Post → sign in required

Signed-in user ———→ all routes allowed

Public Routes vs Protected Routes

- The new RootNavigator provides two versions:
 - Version 1: everything except /auth requires sign-in.
 - Version 2: guests can also view the Home tab; everything else still requires sign-in.
- isProtectedRoute controls where the navigation goes to.
 - In Version 1, it only needs to check if it is /auth route
 - In Version 2, it checks if it is neither /auth nor / (tabs)/undefined (home page verified by console logging)

The new RootNavigator (root/_layout.tsx)

```
useEffect(() => {
  if (loading) return;

  const onAuthScreen = segments[0] === "auth";

  // Option 1: everything except /auth requires sign-in (original behavior).
  const isProtectedRoute = !onAuthScreen;

  // Option 2: guests can also view the Home tab; everything else still requires sign-in.
  // const onHomeScreen = segments[0] === "(tabs)" && segments[1] === undefined;
  // const isProtectedRoute = !onAuthScreen && !onHomeScreen;

  if (!user && isProtectedRoute) {
    router.replace("/auth");
  } else if (user && onAuthScreen) {
    router.replace("/");
  }
}, [user, loading, segments]);
```

Public Routes vs Protected Routes

- We also need to modify the Home Page to disable the + button for unauthenticated users.

The Home Page (index.tsx)

```
// useAuth() gives us the current user from AuthContext.  
// user is guaranteed non-null here - _layout.tsx redirects to /auth if not signed in.  
const { user } = useAuth();  
const authorName = user?.displayName ?? user?.email ?? "Anonymous";  
  
// Option 1: create post always visible.  
//const canPost = true;  
  
// Option 2: guests can view Home but can't create posts.  
const canPost = !!user;
```

```
{canPost && (  
  <TouchableOpacity style={styles.fab} onPress={() => setFormVisible(true)}>  
    <Text style={styles.fabIcon}>+</Text>  
  </TouchableOpacity>  
)}
```

ASSIGNMENT 8

Authentication



Give your app real users.

YOUR TASKS

- Implement email/password auth with Firebase. Persist the session and show the signed-in user's email/name
- Add a Delete button to each post card that is only visible when the signed-in user is the author of that post. Tapping it deletes the post from Firestore.
- Create a "My Posts" tab — new tab showing only the signed-in user's posts, using `useAuth()` to filter

DELIVERABLE

A Firebase-authenticated app where users can sign in with email/password, stay signed in after restart, see their email/name, delete only their own posts from Firestore, and view a new “My Posts” tab showing only posts they created.

EST. EFFORT

≈ 4–5 hours